

EDGE INTELLIGENCE

PRADHEESH RAJ R

25MML0023

```
import os
import cv2
import pickle
import kagglehub
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models
from sklearn.model_selection import train_test_split

# 1. Download Dataset
path = kagglehub.dataset_download("dataclusterlabs/indian-number-plates-dataset")

# 2. Define CNN Architecture (Character Recognizer)
def create_cnn_model(num_classes=36): # 10 digits + 26 letters
    model = models.Sequential([
        layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 1)),
        layers.MaxPooling2D((2, 2)),
        layers.Conv2D(64, (3, 3), activation='relu'),
        layers.MaxPooling2D((2, 2)),
        layers.Flatten(),
        layers.Dense(128, activation='relu'),
        layers.Dense(num_classes, activation='softmax')
    ])
    model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
    return model

# 3. Preprocessing Functions
def segment_characters(plate_img):
    """Detects and crops individual characters from the plate bounding box."""
    gray = cv2.cvtColor(plate_img, cv2.COLOR_BGR2GRAY)
    blur = cv2.GaussianBlur(gray, (5, 5), 0)
    thresh = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)[1]

    # Find contours within the plate
    contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    char_images = []
```

```
# 4. Main Training Pipeline
def run_pipeline():
    # Placeholder for loading your specific bounding box annotations
    # Use [OpenCV Slicing](https://docs.opencv.org) to isolate numbers
    # Example: plate_roi = image[y:y+h, x:x+w]

    print("Building model...")
    model = create_cnn_model()

    # Generate mock data for demonstration (Replace with segments from all_images)
    X = np.random.rand(100, 32, 32, 1)
    y = np.random.randint(0, 36, 100)

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

    print("Training Deep Learning model...")
    history = model.fit(X_train, y_train, epochs=5, validation_data=(X_test, y_test), verbose=1)

    # Calculate Final Accuracy
    loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
    print(f"\nFinal Model Accuracy: {accuracy * 100:.2f}%")

    # 5. Pickle the Model
    # Note: Keras models are best saved as .h5, but we can pickle parameters
    model_export = {
        "weights": model.get_weights(),
        "accuracy": accuracy,
        "classes": "0-9, A-Z"
    }
}
```

```
print(f"Model and accuracy results pickled to: {os.path.abspath(pickle_file)}")

if __name__ == "__main__":
    run_pipeline()

Building model...
Training Deep Learning model...
Epoch 1/5
3/3 ————— 2s 145ms/step - accuracy: 0.0000e+00 - loss: 3.6011 - val_accuracy: 0.0000e+00 - val_loss: 3.5473
Epoch 2/5
3/3 ————— 0s 49ms/step - accuracy: 0.0586 - loss: 3.5110 - val_accuracy: 0.1000 - val_loss: 3.5538
Epoch 3/5
3/3 ————— 0s 47ms/step - accuracy: 0.0688 - loss: 3.4281 - val_accuracy: 0.1000 - val_loss: 3.6056
Epoch 4/5
3/3 ————— 0s 48ms/step - accuracy: 0.0922 - loss: 3.3756 - val_accuracy: 0.1000 - val_loss: 3.6103
Epoch 5/5
3/3 ————— 0s 48ms/step - accuracy: 0.0531 - loss: 3.3727 - val_accuracy: 0.1000 - val_loss: 3.6019

Final Model Accuracy: 10.00%
Model and accuracy results pickled to: /kaggle/working/indian_plate_number_model.pkl
```